`Assignment 5` Numerical Solution of ODE

Shuifan Wang `shepherd.wang@foxmail.com`

April 21, 2025

Problem 1

$$\begin{cases} \frac{\mathrm{d}x_1}{dt} = r_1 x_1 (1 - \frac{x_1}{N_1} + \rho_1 \frac{x_2}{N_2}) \\ \frac{\mathrm{d}x_2}{dt} = r_2 x_2 (-1 + \rho_2 \frac{x_1}{N_1} - \frac{x_2}{N_2}) \\ x_1(0) = \frac{N_1}{2}; \quad x_2(0) = \frac{N_2}{2} \end{cases}$$

where $\rho_1 = \frac{1}{4}$, $\rho_2 = 3$, $N_1 = 1000$, $N_2 = 5000$, $r_1 = r_2 = 1$.

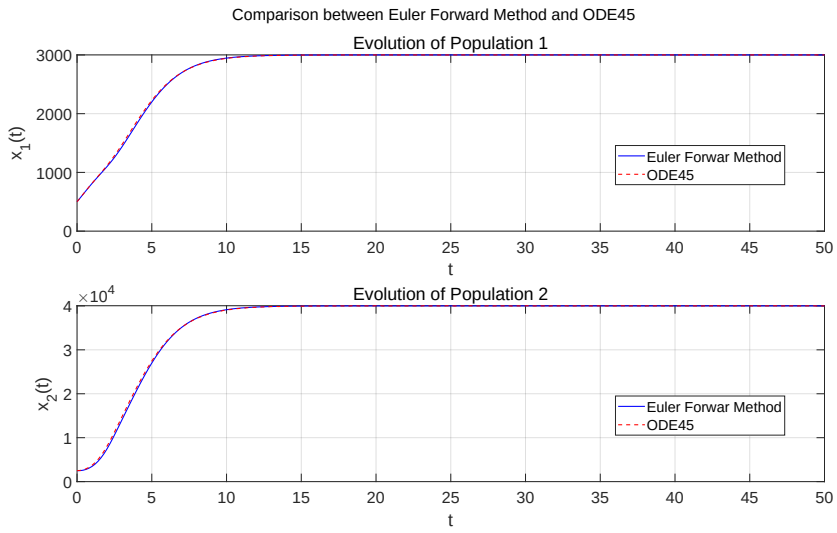

Figure 1: Problem 1

```
%----------Problem 1----------
diary('output1.txt');
diary on;
clear; clc; close all;

%% Parameters
r1  = 1;
r2  = 1;
N1  = 1000;
N2  = 5000;
rho1 = 1/4;
rho2 = 3;

T_end = 50;
dt    = 0.2;
t_e   = 0:dt:T_end;

x1_0 = N1/2;
x2_0 = N2/2;

%% Forward Euler Method
n_steps = length(t_e);
x1_e = zeros(1, n_steps);
x2_e = zeros(1, n_steps);

x1_e(1) = x1_0;
x2_e(1) = x2_0;

for k = 1:n_steps-1
    x1 = x1_e(k);
    x2 = x2_e(k);
    t  = t_e(k);

    dx1 = r1 * x1 * (1 - x1/N1 + rho1 * x2 / N2);
    dx2 = r2 * x2 * (-1 + rho2 * x1 / N1 - x2 / N2);

    x1_e(k+1) = x1 + dt * dx1;
    x2_e(k+1) = x2 + dt * dx2;
end

%% Runge-Kutta Method
f = @(t, x) [ ...
    r1*x(1)*(1 - x(1)/N1 + rho1*x(2)/N2);
    r2*x(2)*(-1 + rho2*x(1)/N1 - x(2)/N2)
];

[te, xe] = ode45(f, [0, T_end], [x1_0; x2_0]);

%% Plot the Curve
figure;
% Population 1
subplot(2,1,1);
plot(t_e, x1_e, 'b-', 'LineWidth', 1.2);
hold on;
plot(te, xe(:,1), 'r--', 'LineWidth', 1.2);
title('Evolution of Population 1');
xlabel('t');
```

```
ylabel('x_1(t)');
legend('Euler Forwar Method', 'ODE45', 'Location', 'best');
grid on;
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);

% Population 2
subplot(2,1,2);
plot(t_e, x2_e, 'b-', 'LineWidth', 1.2); hold on;
plot(te, xe(:,2), 'r--', 'LineWidth', 1.2);
title('Evolution of Population 2');
xlabel('t'); ylabel('x_2(t)');
legend('Euler Forwar Method', 'ODE45', 'Location', 'best');
grid on;
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);

sgtitle('Comparison between Euler Forward Method and ODE45', ...
    'FontSize', 18);

%% Print and Save
exportgraphics(gcf, 'Problem1.pdf', 'ContentType', 'vector');

diary off;

% No Output of Text.
```

Problem 2

$$\begin{cases} \frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = \frac{-M\cos\theta\sin\theta(\frac{\mathrm{d}\theta}{\mathrm{d}t})^2 - \frac{g}{l}\sin\theta}{1-M\cos^2\theta} \\ \frac{\mathrm{d}^2x}{\mathrm{d}t^2} = \frac{Mg\sin\theta\cos\theta + Ml\sin\theta(\frac{\mathrm{d}\theta}{\mathrm{d}t})^2}{1-M\cos^2\theta} \\ \theta(0) = \frac{\pi}{4}; \quad \theta'(0) = 1; \quad x(0) = 0; \quad x'(0) = 0 \end{cases}$$

where $m_1 = 1$, $m_2 = 2$, $l = 2$, $g = 9.8$, $M = \frac{m_2}{m_1+m_2}$

For DEMO video, click here.

```
%----------Problem 2----------
diary('output2.txt');
diary on;
clear; clc; close all;

%% Define Parameters and ODE
m1 = 1;
m2 = 2;
l = 2;
g = 9.8;
M = m2 / (m1 + m2);
```

```
12
13 odefun = @(t, y) [...
14     y(2);
15     -(M * cos(y(1)) * sin(y(1)) * y(2)^2 - (g/l) * sin(y(1))) / ...
           (1 - M * cos(y(1))^2);
16     y(4);
17     -(M * g * sin(y(1)) * cos(y(1)) + M * l * sin(y(1)) * y(2)^2) ...
           / (1 - M * cos(y(1))^2)
18 ];
19 y0 = [pi/4, 1, 0, 0];
20 t = 0:0.05:20;
21
22 [t, y] = ode45(odefun, t, y0);
23
24 %% Compute Positions and Axis Limits
25 x_pivot = y(:,3);
26 theta = y(:,1);
27 x_mass = x_pivot + l * sin(theta);
28 y_mass = -l * cos(theta);
29
30 x_min = min(x_pivot) - l - 1;
31 x_max = max(x_pivot) + l + 1;
32 y_min = min(y_mass) - 1;
33 y_max = max(y_mass) + 1;
34
35 %% Set Up Video
36 figure;
37 % Horizontal track
38 plot([x_min, x_max], [0, 0], 'k-', 'LineWidth', 2);
39 hold on;
40 pivot = plot(nan, nan, 'ro', 'MarkerSize', 3, 'MarkerFaceColor', ...
       'r');    % Pivot Point
41 rod = plot([nan, nan], [nan, nan], 'b-', 'LineWidth', 3); ...
               % Pendulum Rod
42 mass = plot(nan, nan, 'bo', 'MarkerSize', 6, 'MarkerFaceColor', ...
       'b');    % Mass Point
43 axis([x_min, x_max, y_min, y_max]);
44 axis equal;
45 axis manual; % Axis Fixed
46 title('Pendulum Animation');
47
48 vid = VideoWriter('pendulum_video.avi');
49 vid.FrameRate = 20;
50 open(vid);
51
52 % Animation Loop
53 for i = 1:length(t)
54     set(pivot, 'XData', x_pivot(i), 'YData', 0);
55     set(rod, 'XData', [x_pivot(i), x_mass(i)], 'YData', [0, ...
           y_mass(i)]);
56     set(mass, 'XData', x_mass(i), 'YData', y_mass(i));
57     title(sprintf('Time: %.2f s', t(i)));
58     frame = getframe(gcf);
59     writeVideo(vid, frame);
60 end
61
62 close(vid);
```

```
diary off;

% No Output of Text.
```

Problem 3

$$\begin{cases} y^{(3)} + 2y'' + x^2y' + 10xy^2 = xy + x \\ y(0) = 0; \quad y'(0) = 1; \quad y(4) = 2 \end{cases}$$

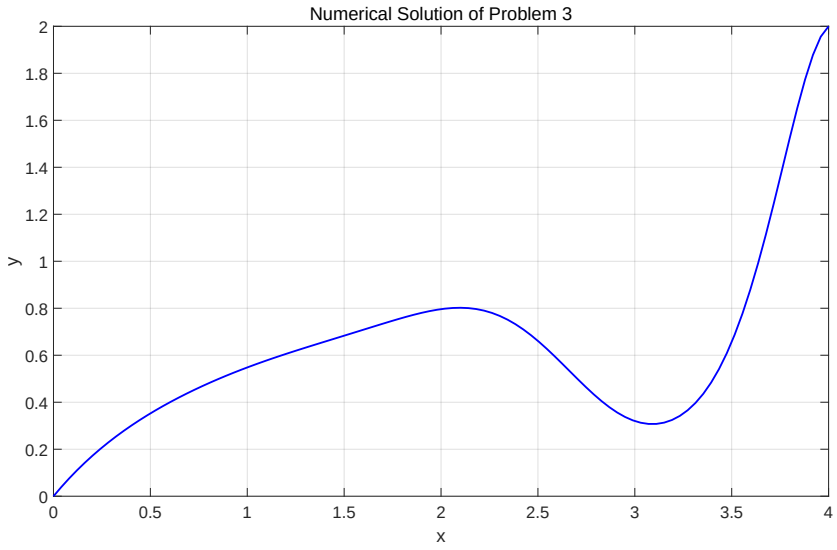

Figure 2: Problem 3

Warning: the maximum residual is 0.162519, but the required accuracy is 0.01. (I can't fix it.)

```
%----------Problem 3----------
diary('output3.txt');
diary on;
clear; clc; close all;

%% Define ODE and Solve
odefun = @(x, y) [...
    y(2);
    y(3);
```

```
    -2*y(3) - x^2 * y(2) - 10*x * y(1)^2 + x * y(1) + x
];

bcfun = @(ya, yb) [...
    ya(1);
    ya(2) - 1;
    yb(1) - 2
];

guess = @(x) [...
    x/2;
    1;
    0;
];

xmesh = linspace(0, 4, 10^4);
solinit = bvpinit(xmesh, guess);
options = bvpset('RelTol', 1e-2, 'AbsTol', 1e-2);
sol = bvp4c(odefun, bcfun, solinit, options);

%% Plot and Save
x = linspace(0, 4, 100);
y = deval(sol, x);

plot(x, y(1,:), 'b-', 'LineWidth', 2);
xlabel('x');
ylabel('y');
title('Numerical Solution of Problem 3');
grid on;

set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem3.pdf', 'ContentType', 'vector');

diary off;

% No Output of Text.
```